Sujets de TP – automne

1. Manipulation des formes de base

1) Définition d'un cube avec des GL_TRIANGLE_STRIP

```
void ViewerEtudiant::init_cube(){
static float pt[8][3] = { {-1,-1,-1}, {1,-1,-1}, {1,-1,1}, {-1,-1,1}, {-1,1,-1}, {1,1,-1}, {1,1,1}, {-1,1,1} };
static int f[6][4] = { {0,1,2,3}, {5,4,7,6}, {2,1,5,6}, {0,3,7,4}, {3,2,6,7}, {1,0,4,5} };
static float n[6][3] = { {0,- 1,0}, {0,1,0}, {1,0,0}, {-1,0,0}, {0,0,1}, {0,0,-1} };
m_cube = Mesh(GL_TRIANGLE_STRIP);
for (int i=0; i<6; i++) // i = numéro de la face
   {
   // La normale à la face
   m_cube.normal(n[i][0], n[i][1], n[i][2]);
   // Les 4 sommets de la face
   m_cube.texcoord(0,0);
   m_cube.vertex( pt[ f[i][0] ][0], pt[ f[i][0] ][1], pt[ f[i][0] ][2] );
   m_cube.texcoord(1,0);
   m_cube.vertex( pt[ f[i][1] ][0], pt[ f[i][1] ][1], pt[ f[i][1] ][2] );
   m_cube.texcoord(0,1);
   m_cube.vertex( pt[ f[i][3] ][0], pt[ f[i][3] ][1], pt[ f[i][3] ][2] );
   m_cube.texcoord(1,1);
   m_cube.vertex( pt[ f[i][2] ][0], pt[ f[i][2] ][1], pt[ f[i][2] ][2] );
   m_cube.restart_strip(); // Demande un nouveau strip
   }
};
```

```
void ViewerEtudiant::draw_cube(const Transform& T)
{
gl.model(T);
gl.draw(m_cube);
}
```

Définition des formes de base : cylindre, cône, sphère.

Disque du cylindre :

```
//initialisation du disque
void ViewerEtudiant::init_disque(){

// Variation de l'angle de 0 à 2π
const int div = 25;
float alpha;
float step = 2.0 * M_PI / (div);
// Choix primitive OpenGL
m_disque = Mesh( GL_TRIANGLE_FAN );
m_disque.normal( Vector(0,1,0) ); // Normale à la surface
m_disque.vertex( Point(0,0,0) ); // Point du centre du disque
// Variation de l'angle de 0 à 2π
for (int i=0; i<=div; ++i)
{
alpha = i * step;
m_disque.normal( Vector(0,1,0) );
m_disque.vertex( Point(cos(alpha), 0, sin(alpha)) );

}
}
```

```
void ViewerEtudiant::draw_disque(const Transform& T)
{

gl.model(T);
gl.draw(m_disque);
}
```

Cône :

```
void ViewerEtudiant::init_cone(){
// Variation de l'angle de 0 à 2π
const int div = 25;
float alpha;
float step = 2.0 * M_PI / (div);
// Choix de la primitive OpenGL
m_cone = Mesh(GL_TRIANGLE_STRIP);

for (int i=0;i<=div;++i)
{
alpha = i * step; // Angle varie de 0 à 2π
m_cone.normal(Vector( cos(alpha)/sqrtf(2.f),0,sin(alpha)/sqrtf(2.f) ));
m_cone.texcoord(i/div, 1);
m_cone.vertex( Point( cos(alpha), 0, sin(alpha) ));
m_cone.normal(Vector( cos(alpha)/sqrtf(2.f),1.f/sqrtf(2.f),sin(alpha)/sqrtf(2.f) ));
m_cone.texcoord(i/div, 0);
m_cone.vertex( Point(0, 1, 0) );
}

}
```

```
void ViewerEtudiant::draw_cone(const Transform& T){
    gl.model(T);
    gl.draw(m_cone);

}
```

Cylindre :

```
void ViewerEtudiant::init_cylindre(){
int i;
const int div = 25;
float alpha;
float step = 2.0 * M_PI / (div);
// Choix primitive OpenGL
m_cylindre = Mesh(GL_TRIANGLE_STRIP);

for (int i=0; i<=div; ++i)
{
// Variation de l'angle de 0 à 2π
alpha = i * step;
m_cylindre.normal( Vector(cos(alpha), 0, sin(alpha)) );
m_cylindre.texcoord(i/div, 1);
m_cylindre.vertex( Point(cos(alpha), -1, sin(alpha)) );

m_cylindre.normal( Vector(cos(alpha), 0, sin(alpha)) );
m_cylindre.texcoord(i/div, 0);
m_cylindre.vertex( Point(cos(alpha), 1, sin(alpha)) );
}

}
```

```
void ViewerEtudiant::draw_cylindre(const Transform& T){
gl.model(T);
gl.draw(m_cylindre);

// Disque du haut
Transform Tch = T * Translation( 0, -1, 0);
gl.model( Tch );
gl.draw( m_disque);
// Disque du bas
Transform Tcb = T * Translation( 0, 1, 0)
* Rotation( Vector(1,0,0), 180);
gl.model( Tcb );
gl.draw( m_disque);
}
```

Sphère :

```
void ViewerEtudiant::init_sphere(){

// Variation des angles alpha et beta
const int divBeta = 16;
const int divAlpha = divBeta/2;
int i,j;
float beta, alpha, alpha2;
// Choix de la primitive OpenGL
m_sphere = Mesh(GL_TRIANGLE_STRIP);

// Variation des angles alpha et beta
for(int i=0; i<divAlpha; ++i)
{
alpha = -0.5f * M_PI + float(i) * M_PI / divAlpha;
alpha2 = -0.5f * M_PI + float(i+1) * M_PI / divAlpha;
for(int j=0; j<=divBeta; ++j)
{
beta = float(j) * 2.f * M_PI / (divBeta);
m_sphere.normal( Vector(cos(alpha)*cos(beta), sin(alpha), cos(alpha)*sin(beta)) );
m_sphere.texcoord(beta/(2*M_PI), 0.5+alpha/M_PI);
m_sphere.vertex( Point(cos(alpha)*cos(beta), sin(alpha), cos(alpha)*sin(beta)) );
m_sphere.normal( Vector(cos(alpha2)*cos(beta), sin(alpha2), cos(alpha2)*sin(beta)));
m_sphere.texcoord(beta/(2*M_PI), 0.5+alpha2/M_PI);
m_sphere.vertex( Point(cos(alpha2)*cos(beta), sin(alpha2), cos(alpha2)*sin(beta)) );
} // boucle sur les j, angle beta, dessin des sommets d'un cercle
m_sphere.restart_strip(); // Demande un nouveau strip
} // boucle sur les i, angle alpha, sphère = superposition de cercles


}
```

---

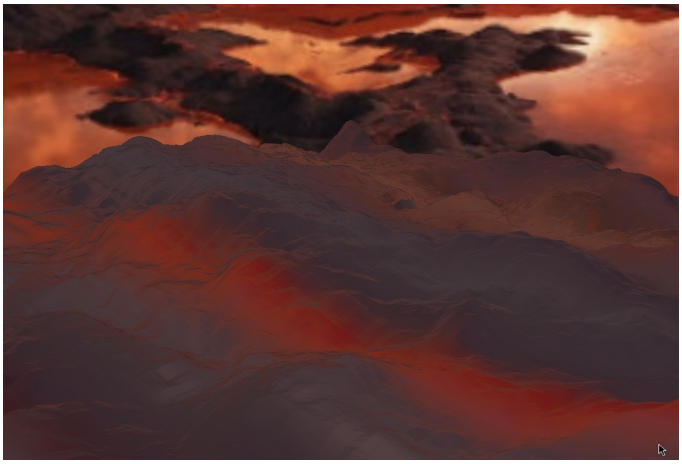
2. Affichage à l'aide de transformations géométriques :

```
void ViewerEtudiant::draw_avion_base(const Transform& T){
    gl.texture(avion_tex);
    gl.model(T *Translation(0,-1,0) * Scale(2,1,1));
    gl.draw(m_sphere);
    gl.texture(avion_tex);
    gl.model(T * Translation(0,-1,6) *  Scale(2,1,1));
    gl.draw(m_sphere);
    gl.texture(avion_tex);
    gl.model(T * Translation(0,0,3) * Scale(6,1,1));
    gl.draw(m_sphere);
    gl.texture(avion_tex);
    gl.model(T * Translation(0,0,3) * Scale(0.6,0.3,5));
    gl.draw(m_cube);
    gl.texture(avion_tex);
    gl.model(T * Translation(-4.5,0,3) * Rotation( Vector(0,0,1), 40) * Scale(0.7,2,0.3));
    gl.draw(m_cone);

}

void ViewerEtudiant::draw_avion(const Transform& T){
  draw_avion_base(T * Scale(0.6,0.6,0.6));

}
void ViewerEtudiant::draw_sphere(const Transform& T){

    gl.model(T);
    gl.draw(m_sphere);
}
```

3.Terrain, texture, billboard (arbre) et cubemap :
3.1.Définition d'un terrain :

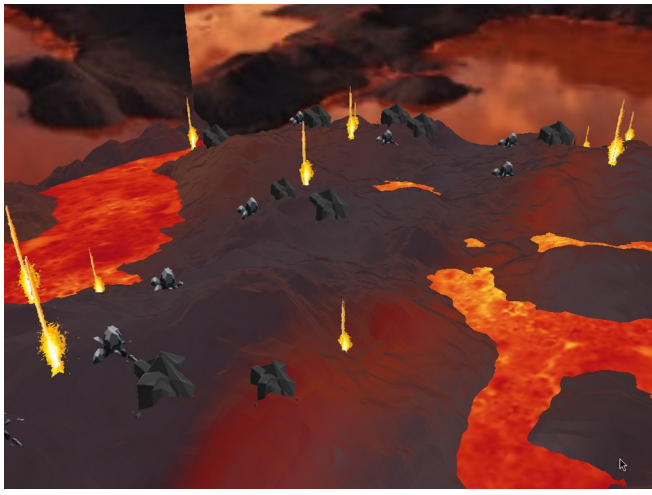
```
//normal du terrain

Vector terrainNormal(const Image& im, const int i, const int j){
// Calcul de la normale au point (i,j) de l'image
int ip = i-1;
int in = i+1;
int jp = j-1;
int jn = j+1;
Point a( ip, im(ip, j).r, j );
Point b( in, im(in, j).r, j );
Point c( i, im(i, jp).r, jp );
Point d( i, im(i, jn).r, jn );
Vector ab = normalize(b - a);
Vector cd = normalize(d - c);
Vector n = cross(ab,cd);
return -n;
}


//initialisation terrain

void ViewerEtudiant::init_terrain(const Image& im){
m_terrain = Mesh(GL_TRIANGLE_STRIP); // Choix primitive OpenGL

for(int i=1;i<im.width()-2;++i){ // Boucle sur les i
    for(int j=1;j<im.height()-1;++j){ // Boucle sur les j
        m_terrain.normal( terrainNormal(im, i+1, j) );
        m_terrain.texcoord(float (i+1)/float(im.width()-2) ,j);
        m_terrain.vertex( Point(i+1, 25.f*im(i+1, j).r, j) );
        m_terrain.normal( terrainNormal(im, i, j) );
        m_terrain.texcoord(float(i)/float(im.height()-1),j);
        m_terrain.vertex( Point(i, 25.f*im(i, j).r, j) );
        }
    m_terrain.restart_strip(); // Affichage en triangle_strip par bande
    }
}
```

```
void ViewerEtudiant::draw_terrain(const Transform &T){
gl.texture(terrain_texture);
gl.model( T );
gl.draw( m_terrain );
}
```



3.2. Ajout (d'arbres) jets de lave et roches sur le terrain :

```
//initialisation des coordonnées des jets de laves
void ViewerEtudiant::init_coord_jet(const Image& im)
{
for (int i=0; i<=130;i++){

tab_nature[i].x = rand() % im.width();
tab_nature[i].z = rand() % im.height();


tab_nature[i] . y = 25.f * im(tab_nature[i].x, tab_nature[i].z).r;
tab_text[i] =  rand() % 5;

}}

//Affiche les jets, roches

//Affiche deux quad superposer pour faire les billboards
void ViewerEtudiant::draw_jet(const Transform &T, int tex){
gl.alpha_texture(tex);
gl.model(T * Translation(0,3,0)* Scale(4,4,4));
gl.draw(m_quad);

gl.alpha_texture(tex);
gl.model(T * Translation(0,3,0) * Rotation(Vector(0,1,0),90) * Scale(4,4,4) );
gl.draw(m_quad);
}

//fonction qu'on utilisera pour affiches toutes les billboards jets de différents tailles ainsi que 2 types de roches
void ViewerEtudiant::draw_nature(const Transform& T) {
    Transform TT = Translation(0,-2.5,0);
    Transform Tt = Translation(0,-1.7,0);
     for (int i = 0; i < 130; ++i) {
         // Transformation pour placer chaque arbre à sa position
         Transform arbre_position = T * Translation(tab_nature[i].x, tab_nature[i].y, tab_nature[i].z);
         if (tab_nature[i].y >11.4){
         // Choix aléatoire de texture d'arbre
         //int arbre_type = 0; //rand() % 4;
         switch (tab_text[i]) {
             case 0:
                 draw_jet(arbre_position * Tt* Scale(1.5,1.5,1.5), arbre_texture);
                 break;
             case 1:
                 draw_jet(arbre_position * Scale(2.4,2.4,2.4), arbre_texture2);
                 break;
             case 2:
                 draw_jet(arbre_position *Tt, arbre_texture3);
                 break;
             case 3:
                 draw_jet(arbre_position*TT, arbre_texture4);
                 break;
             case 4:
                draw_jet(arbre_position * TT * Scale(1.5,1.5,1.5),arbre_texture3);
              }
         }
     }
}
```



3.3 Ajout d'un cubemap autour du terrain :

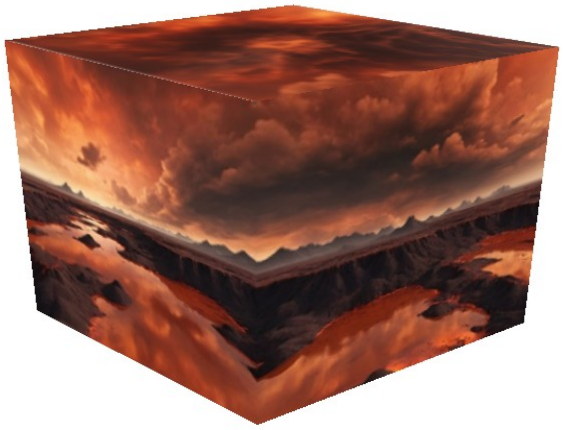
```
void ViewerEtudiant::init_cube1() {
    m_cube1 = Mesh(GL_TRIANGLES); // et non GL_TRIANGLE_STRIP

    // Face avant
    m_cube1.normal(0.0, 0.0, 1.0);
    m_cube1.texcoord(0.25, 0.33);
    m_cube1.vertex(-0.5, -0.5, 0.5);

    m_cube1.normal(0.0, 0.0, 1.0);
    m_cube1.texcoord(0.5, 0.33);
    m_cube1.vertex(0.5, -0.5, 0.5);

    m_cube1.normal(0.0, 0.0, 1.0);
    m_cube1.texcoord(0.5, 0.66);
    m_cube1.vertex(0.5, 0.5, 0.5);

    m_cube1.normal(0.0, 0.0, 1.0);
    m_cube1.texcoord(0.25, 0.66);
    m_cube1.vertex(-0.5, 0.5, 0.5);

    m_cube1.triangle(0, 1, 2);
    m_cube1.triangle(0, 2, 3);

m_cube1.restart_strip();
    // Face arrière
    m_cube1.normal(0.0, 0.0, -1.0);
    m_cube1.texcoord(0.25, 0.33);
    m_cube1.vertex(-0.5, -0.5, -0.5);

    m_cube1.normal(0.0, 0.0, -1.0);
    m_cube1.texcoord(0.5, 0.33);
    m_cube1.vertex(0.5, -0.5, -0.5);

    m_cube1.normal(0.0, 0.0, -1.0);
    m_cube1.texcoord(0.5, 0.66);
    m_cube1.vertex(0.5, 0.5, -0.5);

    m_cube1.normal(0.0, 0.0, -1.0);
    m_cube1.texcoord(0.25, 0.66);
    m_cube1.vertex(-0.5, 0.5, -0.5);

    m_cube1.triangle(4, 5, 6);
    m_cube1.triangle(4, 6, 7);

m_cube1.restart_strip();
    // Face droite
    m_cube1.normal(1.0, 0.0, 0.0);
    m_cube1.texcoord(0.5, 0.33);
    m_cube1.vertex(0.5, -0.5, -0.5);

    m_cube1.normal(1.0, 0.0, 0.0);
    m_cube1.texcoord(0.75, 0.33);
    m_cube1.vertex(0.5, -0.5, 0.5);

    m_cube1.normal(1.0, 0.0, 0.0);
    m_cube1.texcoord(0.75, 0.66);
```

suite du code 2 :

```
    m_cube1.vertex(0.5, 0.5, 0.5);

    m_cube1.normal(1.0, 0.0, 0.0);
    m_cube1.texcoord(0.5, 0.66);
    m_cube1.vertex(0.5, 0.5, -0.5);

    m_cube1.triangle(8, 9, 10);
    m_cube1.triangle(8, 10, 11);

m_cube1.restart_strip();
    // Face gauche
    m_cube1.normal(-1.0, 0.0, 0.0);
    m_cube1.texcoord(0.0, 0.33);
    m_cube1.vertex(-0.5, -0.5, -0.5);

    m_cube1.normal(-1.0, 0.0, 0.0);
    m_cube1.texcoord(0.25, 0.33);
    m_cube1.vertex(-0.5, -0.5, 0.5);

    m_cube1.normal(-1.0, 0.0, 0.0);
    m_cube1.texcoord(0.25, 0.66);
    m_cube1.vertex(-0.5, 0.5, 0.5);

    m_cube1.normal(-1.0, 0.0, 0.0);
    m_cube1.texcoord(0.0, 0.66);
    m_cube1.vertex(-0.5, 0.5, -0.5);

    m_cube1.triangle(12, 13, 14);
    m_cube1.triangle(12, 14, 15);

m_cube1.restart_strip();
    // Face dessus
    m_cube1.normal(0.0, 1.0, 0.0);
    m_cube1.texcoord(0.25, 0.66);
    m_cube1.vertex(-0.5, 0.5, -0.5);

    m_cube1.normal(0.0, 1.0, 0.0);
    m_cube1.texcoord(0.5, 0.66);
    m_cube1.vertex(0.5, 0.5, -0.5);

    m_cube1.normal(0.0, 1.0, 0.0);
    m_cube1.texcoord(0.5, 1.0);
    m_cube1.vertex(0.5, 0.5, 0.5);

    m_cube1.normal(0.0, 1.0, 0.0);
    m_cube1.texcoord(0.25, 1.0);
    m_cube1.vertex(-0.5, 0.5, 0.5);

    m_cube1.triangle(16, 17, 18);
    m_cube1.triangle(16, 18, 19);

m_cube1.restart_strip();
    // Face dessous
    m_cube1.normal(0.0, -1.0, 0.0);
    m_cube1.texcoord(0.25, 0.0);
    m_cube1.vertex(-0.5, -0.5, -0.5);
```

suite du code 3 + Affichage :

```
    m_cube1.normal(0.0, -1.0, 0.0);
    m_cube1.texcoord(0.5, 0.0);
    m_cube1.vertex(0.5, -0.5, -0.5);

    m_cube1.normal(0.0, -1.0, 0.0);
    m_cube1.texcoord(0.5, 0.33);
    m_cube1.vertex(0.5, -0.5, 0.5);

    m_cube1.normal(0.0, -1.0, 0.0);
    m_cube1.texcoord(0.25, 0.33);
    m_cube1.vertex(-0.5, -0.5, 0.5);

    m_cube1.triangle(20, 21, 22);
    m_cube1.triangle(20, 22, 23);
    m_cube1.restart_strip();
}

 //Affiche du cubemap

 void ViewerEtudiant::draw_cube1(const Transform& T)
{
 gl.alpha_texture(cubemap);
 gl.model(T);
 gl.draw(m_cube1);

}
```



---

4. Animation :

```
//update


int ViewerEtudiant::update( const float time, const float delta )
{
    // time est le temps ecoule depuis le demarrage de l'application, en millisecondes,
    // delta est le temps ecoule depuis l'affichage de la derniere image / le dernier appel a draw(), en millisecondes.


int ts = time/100; // le plus petit le diviseur, l'avion aura plus de vitesse.

int te = int(ts);

int ite = te% m_anim.nb_points();

float poids = ts-te;

int ite_suiv = (te+ 1)%m_anim.nb_points();

int ite_suiv_suiv = (ite_suiv +1)%m_anim.nb_points();

// les points que l'avion devra suivre

Point p0 = m_anim[ite];

Point p1 = m_anim[ite_suiv];

Point p2 = m_anim[ite_suiv_suiv];

Vector pos = Vector(p0) + (Vector(p1)-Vector(p0))*poids;

Vector pos_suiv = Vector(p1) + (Vector(p2)-Vector(p1))*poids;

Vector dir =normalize((pos_suiv -pos));

Vector up (0,1,0);

Vector codir = cross (dir,up);

m_t_avion = Transform(dir,up,codir,pos);


return 0;

}
```

---

5. Monde Virtuelle :
code :
normal Volcan:

```
void ViewerEtudiant::create_vertex_normal_volcan() {
    // Points de la silhouette 2D
    Point volcan_p[volcan_NBPT];
    volcan_p[0] = Point(0, 0, 0);
    volcan_p[1] = Point(2.5, 0.0, 0);
    volcan_p[2] = Point(1.5, 1.0, 0);
    volcan_p[3] = Point(1.0, 1.8, 0);
    volcan_p[4] = Point(0.8, 2.8, 0);
    volcan_p[5] = Point(0.9, 3.5, 0);
    volcan_p[6] = Point(0, 3.3, 0);

    // Boucle sur le nombre de rotations
    for (int i = 0; i < volcan_NBROT; i++) {
        // Angle qui varie de 0 à 2π
        float teta = 2.0 * M_PI * float(i) / volcan_NBROT;

        // Matrice de rotation de l'angle theta autour de l'axe des y
        // en coordonnées homogènes : 4 x 4
        float mat[16] = {
            cos(teta), 0, -sin(teta), 0,
            0, 1, 0, 0,
            sin(teta), 0, cos(teta), 0,
            0, 0, 0, 1
        };

        // Calcul des coordonnées des sommets
        for (int j = 0; j < volcan_NBPT; j++) {
            volcan_v[i][j].x = mat[0] * volcan_p[j].x + mat[1] * volcan_p[j].y + mat[2] * volcan_p[j].z + mat[3] * 1;
            volcan_v[i][j].y = mat[4] * volcan_p[j].x + mat[5] * volcan_p[j].y + mat[6] * volcan_p[j].z + mat[7] * 1;
            volcan_v[i][j].z = mat[8] * volcan_p[j].x + mat[9] * volcan_p[j].y + mat[10] * volcan_p[j].z + mat[11] * 1;
                                }


    // Initialisation à 0 des normales
    for (int i = 0; i < volcan_NBROT; i++) {
        for (int j = 0; j < volcan_NBPT; j++) {
            volcan_vn[i][j] = Vector(0, 0, 0);
        }
    }



    // Calcul des normales
for (int i = 0; i < volcan_NBROT; i++) {
    for (int j = 0; j < volcan_NBPT - 1; j++) {
        Vector a_temp = volcan_v[i][j] - volcan_v[i][j + 1];
        Vector b_temp = volcan_v[i][j] - volcan_v[(i + 1) % volcan_NBROT][j];

        Vector a, b, vntmp;

        // Vérification avant normalisation
        if (length(a_temp) > 1e-6) {
            a = normalize(a_temp);
        } else {
            a = Vector(0, 0, 0);
        }

        if (length(b_temp) > 1e-6) {
            b = normalize(b_temp);
        } else {
            b = Vector(0, 0, 0);
        }

        vntmp = cross(a, b); // Produit vectoriel

        // Vérification des résultats
        if (isnan(vntmp.x) || isnan(vntmp.y) || isnan(vntmp.z)) {
            vntmp = Vector(0, 0, 0);
        }

        // Accumulation des normales
        volcan_vn[i][j] = vntmp + volcan_vn[i][j];
        volcan_vn[(i + 1) % volcan_NBROT][j] = vntmp + volcan_vn[(i + 1) % volcan_NBROT][j];
        volcan_vn[(i + 1) % volcan_NBROT][j + 1] = vntmp + volcan_vn[(i + 1) % volcan_NBROT][j + 1];
        volcan_vn[i][j + 1] = vntmp + volcan_vn[i][j + 1];
    }
}
}

    // Moyenne des normales pour chaque sommet
    for (int i = 0; i < volcan_NBROT; i++) {
        for (int j = 0; j < volcan_NBPT; j++) {
            float q = 4.0f;
            if (j == volcan_NBPT - 1) { // Points du bord
                q = 2.0f;
            }
            volcan_vn[i][j] = volcan_vn[i][j] / q;
        }
    }
}
```

init volcan

```
void ViewerEtudiant::init_volcan() {
    m_volcan = Mesh(GL_TRIANGLES);
    m_volcan.color(1.0,1.0,1.0);



    for (int i = 0; i < volcan_NBROT; i++) {
        for (int j = 0; j < volcan_NBPT - 1; j++) { // Attention boucle de 0 à volcan_NBPT-2
            // Premier triangle
            m_volcan.normal(volcan_vn[i][j]);
                m_volcan.texcoord(0,0);
```

```
            m_volcan.vertex(volcan_v[i][j]);

            m_volcan.normal(volcan_vn[(i + 1) % volcan_NBROT][j + 1]);
            m_volcan.texcoord(1,1);
            m_volcan.vertex(volcan_v[(i + 1) % volcan_NBROT][j + 1]);

            m_volcan.normal(volcan_vn[(i + 1) % volcan_NBROT][j]);
            m_volcan.texcoord(1,0);
            m_volcan.vertex(volcan_v[(i + 1) % volcan_NBROT][j]);

            // Second triangle
            m_volcan.normal(volcan_vn[i][j]);
                                                    m_volcan.texcoord(0,0);
            m_volcan.vertex(volcan_v[i][j]);

            m_volcan.normal(volcan_vn[i][j + 1]);
                m_volcan.texcoord(0,1);
            m_volcan.vertex(volcan_v[i][j + 1]);

            m_volcan.normal(volcan_vn[(i + 1) % volcan_NBROT][j + 1]);
                m_volcan.texcoord(1,1);
            m_volcan.vertex(volcan_v[(i + 1) % volcan_NBROT][j + 1]);
        }
    }
}
```

draw lave :

```
 void ViewerEtudiant::draw_lava(const Transform &T){
gl.texture(sol);

gl.model( T * Translation(0,-1.8,0)*Rotation(Vector(1,0,0),-90) * Scale(87,87,87) );

gl.draw( m_quad );
 }
```

Plusieurs images :